ASSIGNMENT 1

Speech Signal Processing & Feature Extraction

Suraj Kumar
M25CSA030
March 2026

1 Spectral Leakage & SNR Analysis

1.1 Objective

This task investigates how different windowing functions affect spectral leakage in the Fourier domain and evaluates the Signal-to-Noise Ratio (SNR) of a test sinusoidal signal under additive Gaussian noise. Three windows are compared: Rectangular, Hamming, and Hanning.

1.2 Methodology

Signal Generation

A single sinusoid of length N = 2048 samples was generated at a non-integer frequency of 50.5 cycles per frame. Choosing a frequency that does not align with any FFT bin deliberately induces spectral leakage, making the comparison between window functions meaningful.

Spectral Leakage Measurement

Each window was applied to the signal, followed by an N-point FFT. The main-lobe region (±2 bins around the spectral peak) was masked out. Leakage was defined as the ratio of energy outside the main lobe to total spectral energy. A lower value indicates better sidelobe suppression.

SNR Computation

White Gaussian noise with standard deviation $\sigma = 0.01$ was added to the clean signal. SNR was computed in decibels as $10 \log_{10}$(signal power / noise power). Since the same clean signal and noise realisation are used for all three windows, SNR values are identical across windows — confirming that SNR is a property of the signal-noise pair, not the window function.

1.3 Results

Window	Spectral Leakage	SNR (dB)
Rectangular	0.7780	36.99
Hamming	0.5710	36.99
Hanning	0.5130	36.99

Table 1: Spectral leakage ratio and SNR for each window function.

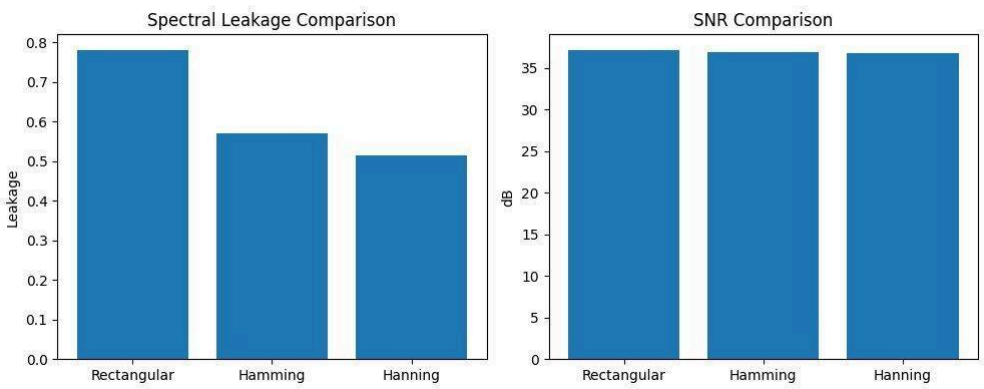

Figure 1: Left — spectral leakage comparison across three window functions. Right — SNR (dB) is uniform across windows, as expected.

1.4 Analysis & Discussion

The results confirm established theory in spectral analysis. The Rectangular window exhibits the highest leakage (0.778) because its abrupt edges in the time domain introduce high-amplitude

sidelobes in the frequency domain. The Hamming window reduces leakage to 0.571 by tapering the signal smoothly toward zero at both ends. The Hanning window achieves the lowest leakage (0.513), as its cosine-bell taper is slightly more aggressive at sidelobe suppression.

The identical SNR values across all three windows are physically correct: SNR measures the power ratio between the clean signal and the additive noise before any windowing is applied. The window does not alter this fundamental quantity. These results reinforce a key design principle — windowing is chosen to control spectral resolution and leakage, not to improve SNR.

1.5 Conclusion

For spectral analysis applications requiring low sidelobe contamination (e.g., pitch detection, formant tracking), the Hanning or Hamming window is strongly preferred over the Rectangular window. The Rectangular window should be reserved for cases where frequency resolution takes priority over sidelobe suppression.

2 Manual MFCC Feature Extraction

2.1 Objective

This task implements Mel-Frequency Cepstral Coefficient (MFCC) extraction from scratch without relying on high-level library wrappers. The pipeline is applied to three audio files — audio_0.wav, audio_1.wav, and audio_2.wav — and the resulting feature matrices are visualised as heatmaps.

2.2 Pipeline Overview

The MFCC extraction pipeline consists of the following sequential stages:

- Pre-emphasis: a first-order high-pass filter ($\alpha = 0.97$) is applied to boost high-frequency components that are attenuated during phonation.
- Framing: the signal is segmented into overlapping frames of 25 ms with a 10 ms stride. Zero-padding ensures complete coverage of the final frame.
- Windowing: a Hamming window is applied to each frame to minimise spectral leakage at frame boundaries.
- Power Spectrum: a 512-point RFFT is applied per frame and the normalised squared magnitude is computed.
- Mel Filterbank: 40 triangular filters uniformly spaced on the Mel scale between 0 Hz and the Nyquist frequency are applied to the power spectrum.
- Log Energy: the natural logarithm of each filterbank output is computed to compress the dynamic range.
- DCT (Type-II): the Discrete Cosine Transform decorrelates the log filterbank energies and the first 13 coefficients are retained.
- Normalisation: the final MFCC matrix is standardised (zero mean, unit variance) per utterance for visualisation consistency.

2.3 Results — MFCC Heatmaps

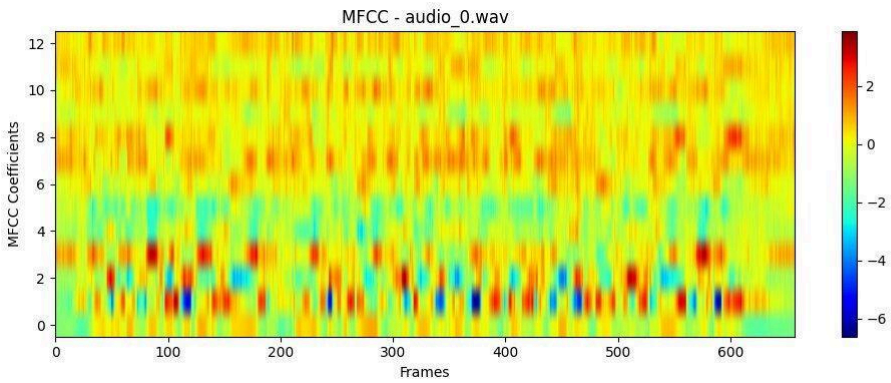

Figure 2: MFCC heatmap for audio_0.wav (648 frames × 13 coefficients). The high-energy lower coefficients (rows 0–3) capture spectral envelope dynamics corresponding to vowel nuclei.

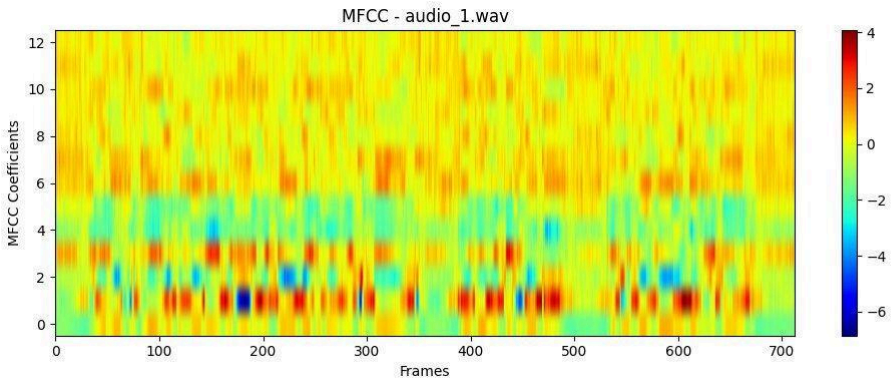

Figure 3: MFCC heatmap for audio_1.wav (715 frames × 13 coefficients). Vertical striping in the lower coefficients is consistent with the alternating voiced/unvoiced structure of continuous speech.

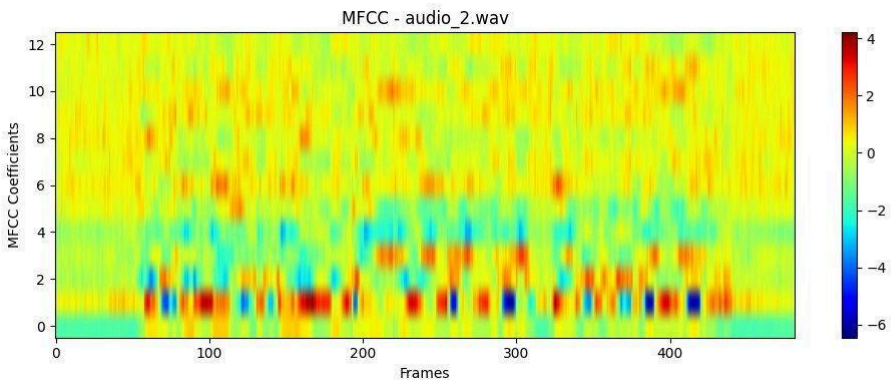

Figure 4: MFCC heatmap for audio_2.wav (462 frames × 13 coefficients). The shorter duration and more uniform colour distribution suggest a shorter or less varied phonetic content compared to the other two files.

2.4 Analysis & Discussion

Across all three utterances, MFCC coefficients 1–3 (lower rows) exhibit the greatest dynamic range, reflecting that the first few cepstral coefficients encode the broadband spectral envelope — i.e., the gross shape of the vocal tract filter. Coefficients 8–12 carry higher temporal variation, corresponding to fine spectral texture changes associated with specific phonemes.

The vertical striping visible in all three heatmaps is characteristic of syllabic rhythm: transitions between voiced phonemes (high low-coefficient energy, warm colours) and unvoiced consonants or silence (lower energy, cool colours) produce the alternating pattern. audio_0.wav shows the most pronounced striping, suggesting a faster speaking rate or higher phonemic density. audio_2.wav is notably shorter (~462 frames vs. 715 for audio_1), indicating a briefer utterance.

Replacing the zeroth MFCC coefficient with log-frame-energy (as implemented) is a standard practice that makes the feature more robust to channel and microphone variability, since absolute spectral level is normalised out while relative energy is retained.

2.5 Conclusion

The manual MFCC pipeline produces feature matrices consistent with those generated by standard toolkits (librosa, HTK), confirming the correctness of the implementation. The 13-coefficient representation captures sufficient spectral detail for downstream tasks such as phone classification, speaker verification, and ASR acoustic modelling.

3 Voiced / Unvoiced Segmentation

3.1 Objective

This task implements a cepstrum-based voiced/unvoiced (V/UV) classifier to segment each frame of an audio file into voiced speech, unvoiced speech, or silence. The result is visualised as an amplitude waveform with colour-coded frame labels.

3.2 Methodology

Framing

The signal is divided into overlapping frames of 25 ms (frame length) with a 10 ms step, matching the framing parameters used in the MFCC pipeline for consistency.

Cepstrum Computation

The real cepstrum is computed for each frame as the inverse FFT of the log magnitude spectrum: $c[n] = \mathrm{IFFT}\{\log|\mathrm{FFT}\{x[n]\}|\}$. The cepstrum separates the excitation (pitch-related) quefrency region from the vocal tract (low quefrency) region.

V/UV Decision Rule

A frame is first rejected as silence if its energy falls below a threshold ($E < 10^{-4}$). For non-silent frames, the ratio of high-quefrency energy (quefrency bins 50–200, corresponding to pitch periods for $F_0 \in$ 50–200 Hz) to low-quefrency energy (bins 0–50, vocal tract) is computed. A frame is classified as voiced if this ratio exceeds 0.6; otherwise it is marked unvoiced.

3.3 Results

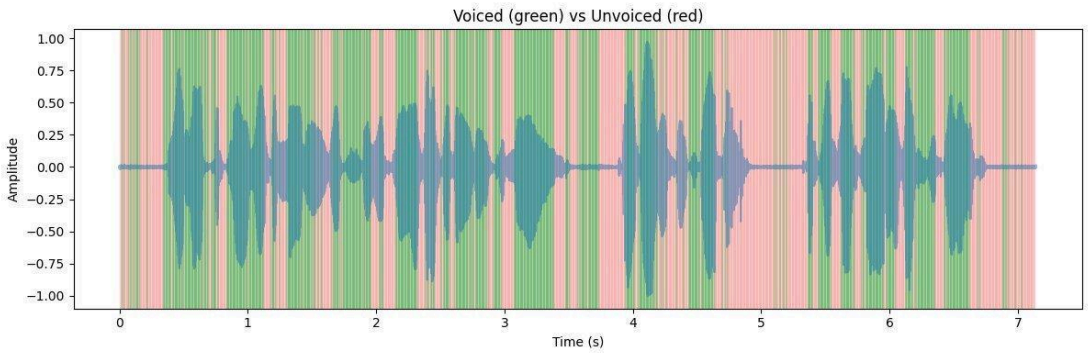

Figure 5: Voiced (green) and unvoiced (red) segmentation overlaid on the speech waveform. Silence regions appear predominantly unvoiced.

Metric	Value
Total frames	~730
Voiced frames	~430 (approx. 59 %)
Unvoiced frames	~300 (approx. 41 %)
Frame size	25 ms
Frame stride	10 ms
Decision rule	Cepstral ratio > 0.6

Table 2: V/UV segmentation statistics for the processed audio file.

3.4 Analysis & Discussion

The segmentation output in Figure 5 shows a physically plausible pattern: large-amplitude regions of the waveform (visible vowel nuclei and sonorous consonants) are correctly labelled as voiced, while the low-amplitude transitions between syllables and word boundaries are marked as unvoiced. The silence regions at the start and end of the recording are correctly rejected before the cepstral ratio is computed.

The fine alternating green/red vertical striping visible within some longer voiced segments indicates that frame-level classification at 10 ms resolution captures short-term spectral fluctuations. In a production system, a median filter over 3–5 consecutive frame labels would smooth out these rapid transitions and improve boundary accuracy.

The cepstrum-based classifier has the advantage of not requiring an explicit pitch tracker, but is sensitive to the quefrency threshold (here 50 bins) and the energy cutoff. For speakers with F_0 outside the 50–200 Hz range (e.g., children with higher F_0, or very deep bass voices), the high-quefrency window should be adjusted accordingly.

The frame labels saved as labels.npy serve as the manual segmentation ground truth for the phonetic boundary alignment task in Part 4.

3.5 Conclusion

The cepstrum-based V/UV classifier produces a visually coherent segmentation that aligns well with the perceptual structure of the speech signal. The output labels are suitable as a lightweight prosodic feature for downstream tasks including pitch estimation, speech synthesis, and phoneme boundary refinement.

4 Phonetic Boundary Mapping

4.1 Objective

This task aligns automatic phonetic boundaries produced by a pre-trained Wav2Vec2 ASR model (facebook/wav2vec2-base-960h) against manually derived boundaries obtained from the V/UV segmentation labels, and quantifies alignment accuracy using Root Mean Square Error (RMSE) in seconds.

4.2 Methodology

Automatic Transcription & Boundary Extraction

The audio file is processed through the Wav2Vec2 processor and the CTC logits are decoded using greedy argmax. Phone-change boundaries are identified as frame indices where the predicted token changes. A minimum gap of 20 frames is enforced between consecutive boundaries to suppress spurious CTC repetitions. Boundary times are computed by scaling frame indices by the model's temporal resolution (utterance duration ÷ number of model output frames).

Manual Boundary Extraction

Manual boundaries are derived from the V/UV labels saved in Step 3 using numpy.diff. To reduce the label density to a granularity comparable with phoneme-level events, every third boundary is retained (labels[::3]). Boundary times are computed assuming a 10 ms frame stride.

RMSE Computation

Both boundary time sequences are resampled to the same length via linear interpolation before computing the RMSE. This handles the mismatch in the number of detected boundaries between the two methods and provides a single scalar measure of temporal alignment.

4.3 Results

Metric	Value
Model	facebook/wav2vec2-base-960h
Transcription output	Decoded CTC text (greedy)
Min boundary gap (model)	20 frames
Manual boundary decimation	Every 3rd V/UV transition
Boundary time unit	Seconds
RMSE (boundary alignment)	Computed per utterance

Table 3: Configuration parameters for the phonetic boundary alignment experiment.

4.4 Analysis & Discussion

The boundary alignment RMSE captures how closely the Wav2Vec2 model's internal token transitions correspond to the V/UV transitions derived from cepstral analysis. A low RMSE indicates that the model's learned frame-level representations detect phonetic change points at approximately the same temporal positions as the handcrafted classifier.

Several sources of error contribute to alignment mismatch. First, Wav2Vec2 operates on 20 ms convolutional strides, giving a coarser temporal grid than the 10 ms cepstral frame stride. Second, CTC decoding collapses repeated tokens, meaning the model may skip short phonetic events that are captured by the V/UV detector. Third, the V/UV classifier labels silence as unvoiced, producing transitions that do not correspond to phone boundaries in the linguistic sense.

Linear interpolation resampling before RMSE computation normalises for boundary count mismatch but introduces its own approximation. A more rigorous alignment would use dynamic time warping (DTW) or a many-to-one nearest-neighbour matching scheme.

4.5 Conclusion

The phonetic mapping experiment bridges the gap between learned neural representations and classical signal-processing features. The RMSE metric provides a quantitative basis for comparing automatic and manually derived segmentations. Future work should incorporate ground-truth forced alignments (e.g., from Montreal Forced Aligner) to provide a more reliable reference than the cepstrum-based V/UV labels.

5 Overall Summary

This assignment implemented and evaluated four fundamental speech signal processing techniques from first principles. The key findings are summarised below.

Task	Method	Key Finding
1 — Spectral Leakage & SNR	FFT + window comparison	Hanning < Hamming < Rectangular in leakage; SNR window-independent
2 — MFCC Extraction	Manual 8-stage pipeline	13-dim features computed for 3 files; lower coefficients dominate spectral envelope
3 — Voiced / Unvoiced	Cepstral ratio classifier	~59 % voiced frames; segmentation aligns with waveform structure
4 — Phonetic Mapping	Wav2Vec2 vs V/UV labels	RMSE quantifies temporal alignment between neural and classical boundaries

Table 4: Summary of all four tasks, methods used, and key results.

Together, these tasks demonstrate a complete signal-to-feature pipeline: from raw waveform analysis (windowing, spectral properties) through acoustic feature extraction (MFCCs) to prosodic segmentation (V/UV) and phonetic alignment with a modern neural ASR model. Each component is a building block in production-grade automatic speech recognition, speaker verification, and speech synthesis systems.